VR OpenGL Project

Jean-Philippe Kleijkers Quentin Lecocq

May 2026

1 Introduction

This project is a 3D space shooter developed using OpenGL. The player evolves on an unknown planet in a hostile space environment and must defend an energy barrier protecting a procedural tree from waves of alien creatures.

The game combines multiple real-time rendering techniques such as dynamic lighting, textured models, reflections, particles, tessellation, procedural generation, and collision physics. The player can freely explore the environment, fight enemies using a laser weapon, and interact with a fully animated world rendered in real time.

The objective of this project was to implement both graphical and gameplay-related features while optimizing performance using modern GPU techniques such as instancing, shaders, and procedural geometry generation. Our repository is available on `https://github.com/jpjp1452/OpenGL_INFO_512_project`.

2 Basic Features

2.1 Lighting and Textures

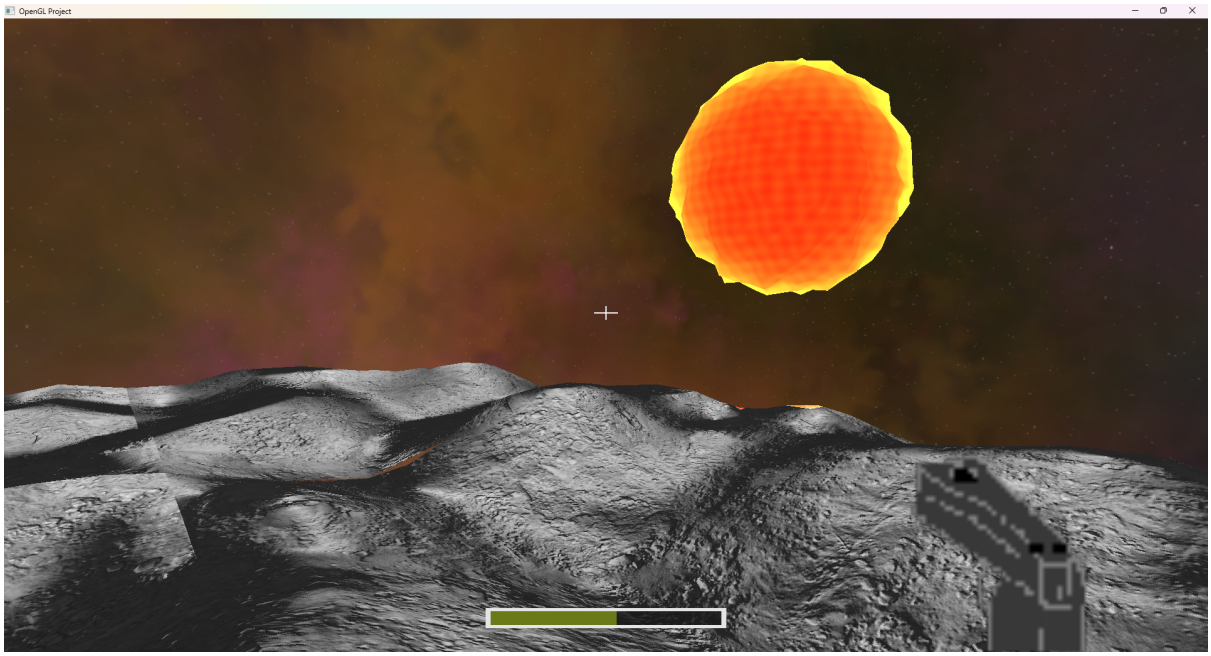

Figure 1: The sun projecting light onto the ground

Our scene is illuminated by the sun, as shown in Figure 1. This lighting system also creates shadows on the ground and darkens some parts of the environment.

To achieve a better rendering quality, we use textures on multiple objects such as the ground, trees, cubemap, asteroids, planet, aliens, and weapons.

2.2 Models

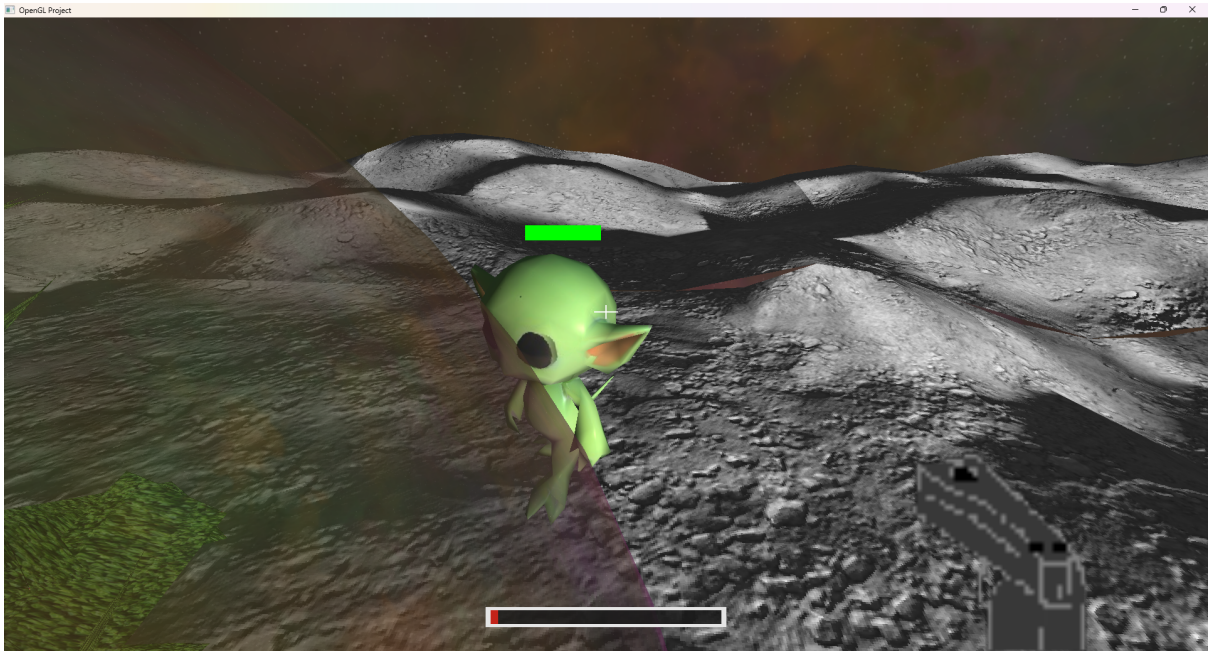

Figure 2: An alien walking toward the barrier

Figure 3: An alien attacking the barrier

We use multiple 3D models within our scene. Models such as `rock.obj` and `small_alien.obj` are textured and integrated into the environment, as shown in Figures 2 and 3. Asteroids are shown in Figure 12.

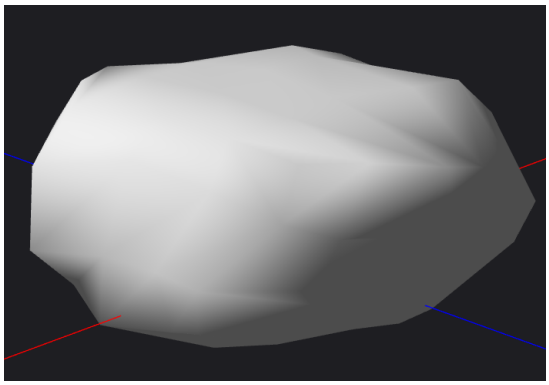

Figure 4: An asteroid as a .obj file

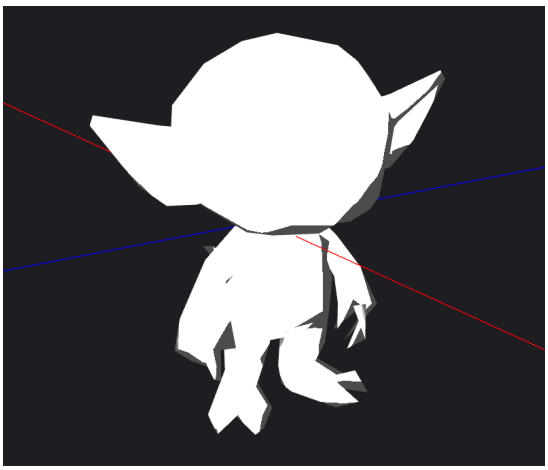

Figure 5: An alien as a .obj file

We also use basic geometric objects such as spheres (Figure 6) and cylinders (Figure 7) to create additional elements like the sun, the planet Mars, and the laser projectiles shown in Figure 9.

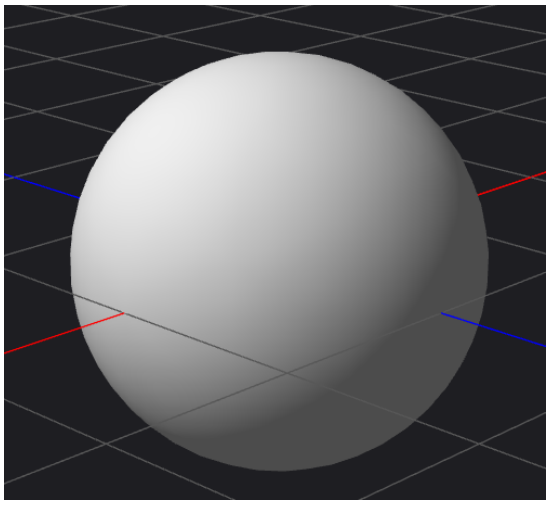

Figure 6: A sphere as a .obj file

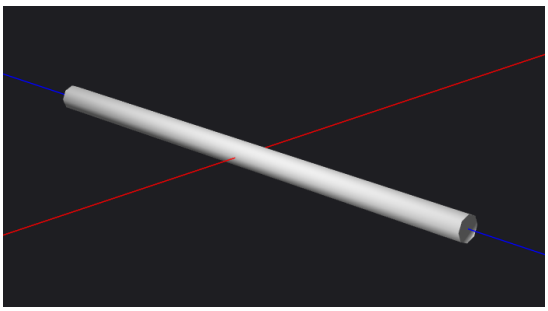

Figure 7: A cylinder as a .obj file

2.2.1 Cubemap

As the name suggests, our game takes place in a space environment. Therefore, we use a space cubemap as the background. The cubemap texture was created using `https://tools.wwwtyro.net/space-3d/index.html`. It allows the player to observe a complete space scene in every direction.

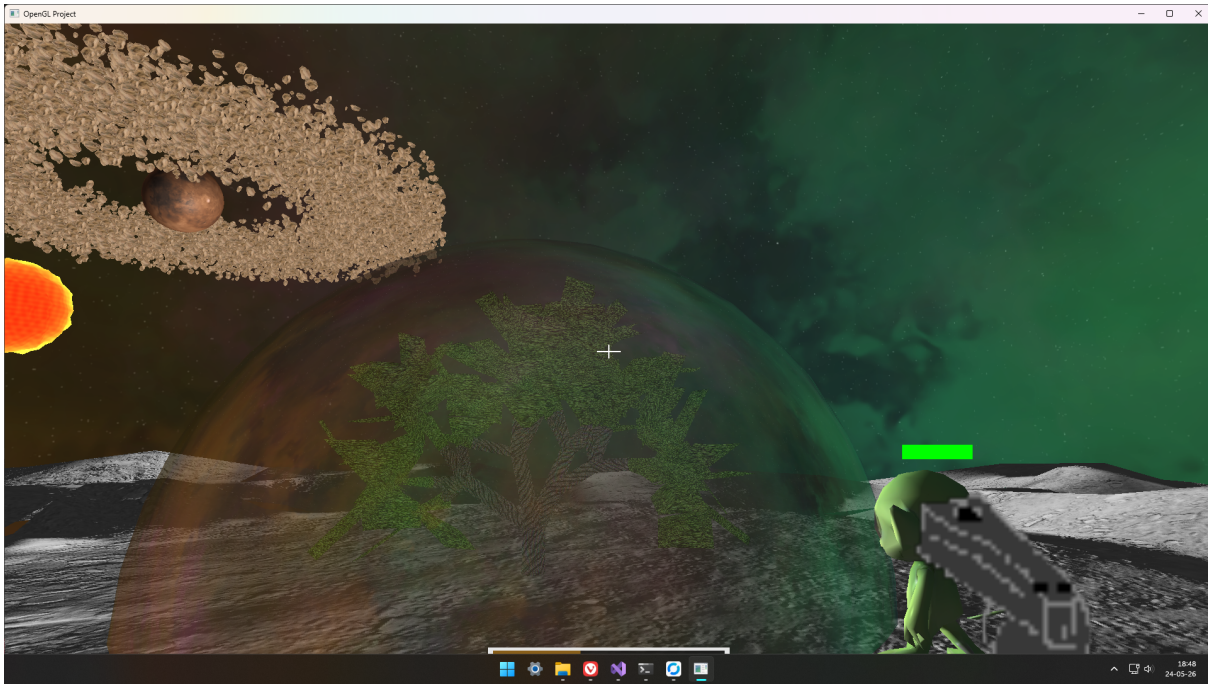

Figure 8: Large view of the space environment

2.2.2 Game Logic and Free Navigation

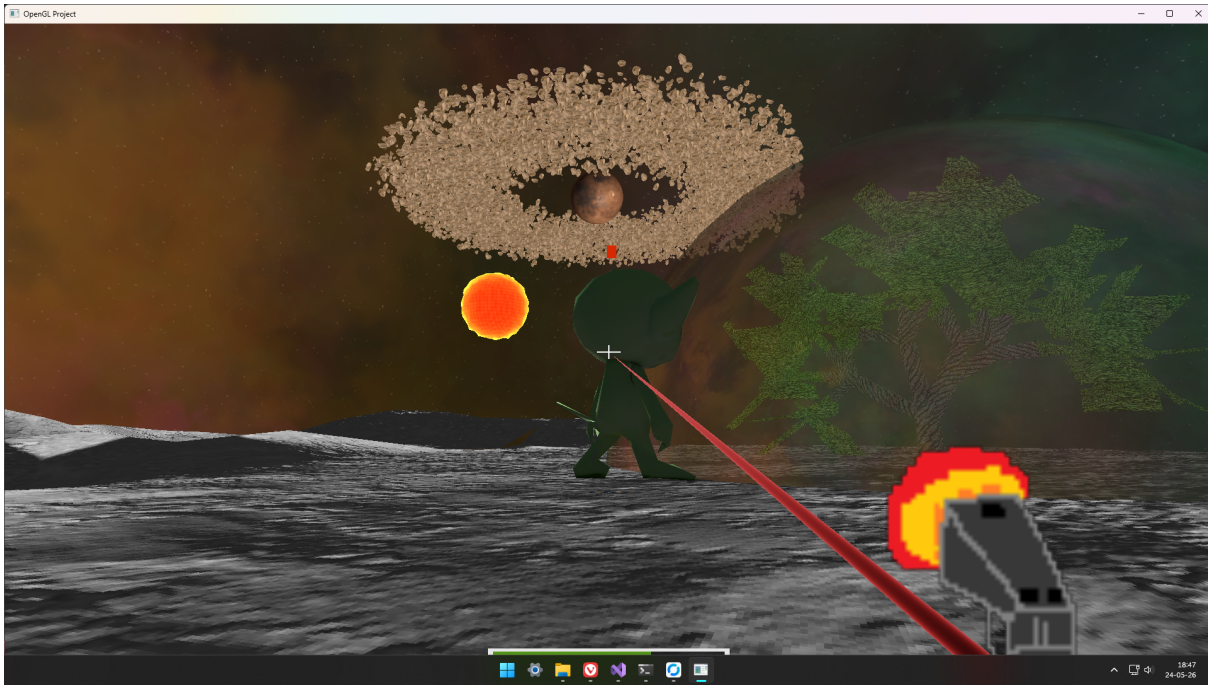

Figure 9: Shooting an alien

The game consists of defending an energy barrier that protects an important procedural tree from invading aliens, as shown in Figure 10.

The player is equipped with a space gun (Figure 9) and evolves on an unknown low-gravity

planet that can be freely explored. When aliens reach the barrier, it loses health points. The game ends when the barrier is completely destroyed.

The player can rotate the camera using the mouse, move using the `ZQSD` keys, and jump using the space bar. Aliens can be eliminated using the left mouse button.

2.3 Moving Objects in the Scene

As shown in the video, aliens continuously move toward the tree. Their leg animations are implemented directly in the vertex shader.

2.4 Reflection and Refraction

The barrier is a reflective sphere that reflects the cubemap texture, as shown in Figure 10. It is also semi-transparent, allowing the player to see the tree inside it.

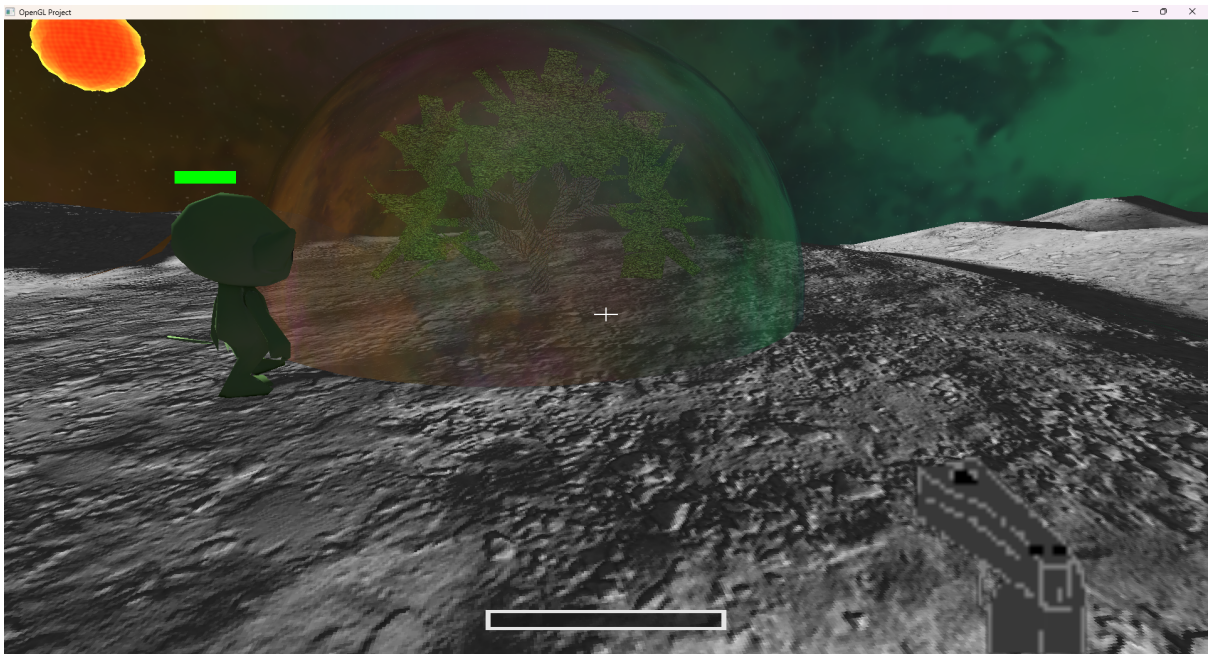

Figure 10: The reflective energy barrier

3 Intermediate Features

3.1 Particles

When a laser shot collides with the ground, sparks and particles are generated at the impact location. Their movement is randomized, and they are updated every frame until they disappear, as shown in Figure 12.

3.2 Instancing

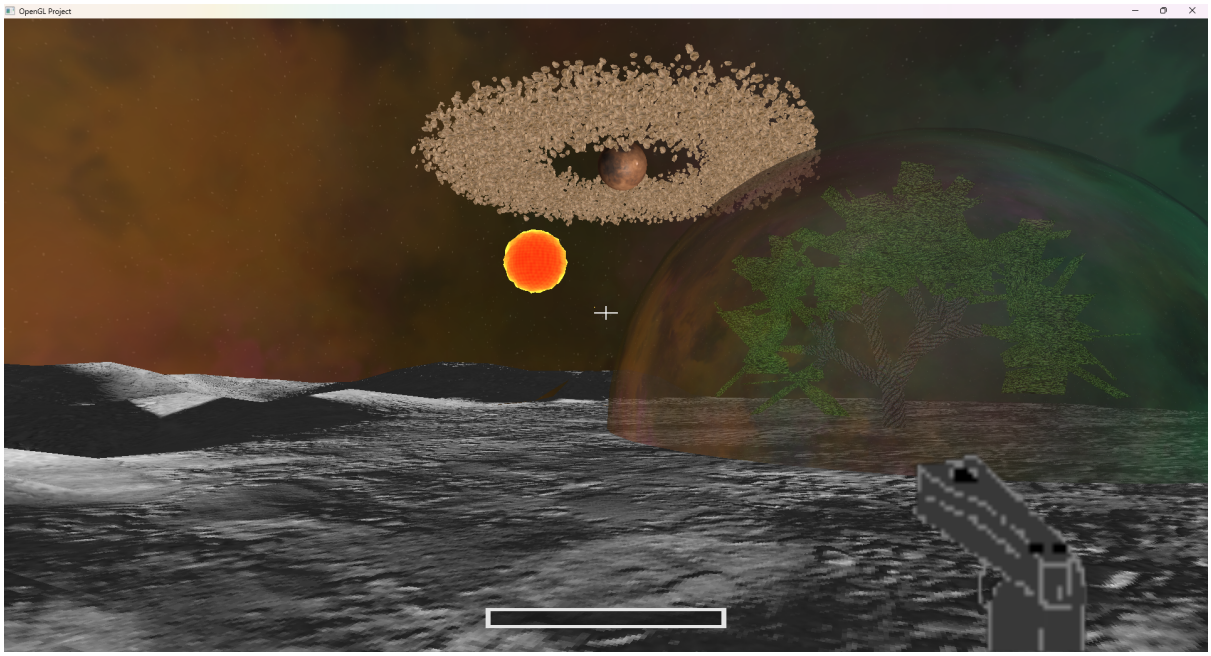

Figure 11: Large view of the space game

For objects generated many times, such as particles (Figure 12) and asteroids (Figure 11), we use instancing. This technique significantly reduces memory usage and improves rendering performance because the GPU processes only a single base mesh.

3.3 Billboarding

As the player moves, we use billboarding so that 2D elements always face the camera. This technique is used for impact particles and the health bars displayed above the aliens.

Billboarding ensures that particle effects always remain visually convincing from the player's point of view while keeping the health bars readable at all times.

4 Advanced Features

4.1 Heightmap and Normal Mapping

For the terrain, we use a heightmap generated with Perlin noise to displace vertices and create a hilly environment. A texture is then applied to the terrain surface.

This texture also contains a grayscale heightmap that is used as a normal map instead of directly modifying the geometry. Additional normals are computed from this texture and combined with the terrain normals to increase visual detail without increasing geometric complexity.

This approach keeps the terrain geometry relatively simple while making CPU-side interpolation and collision calculations easier to perform.

4.2 Collision Detection and Physics

To determine the terrain height under a given point, we use bilinear interpolation.

First, we identify the terrain chunk containing the point. Then, we compute the UV coordinates inside the chunk to retrieve the surrounding heights and interpolate between them.

Using this method, we ensure that the player cannot pass through the terrain, even when gravity is applied.

Aliens move toward their target while continuously adjusting their height according to the terrain below them. They stop moving once they reach the energy barrier.

For lasers and impact positions, we trace the projectile trajectory to detect when the ray goes below the terrain surface, which corresponds to the collision point shown in Figure 12. Particle velocities are then used to animate the explosion effects around the impact area.

Figure 12: Collision between a laser shot and the ground

4.3 3D Mesh Loader

We implemented an improved .obj reader based on the one developed during the exercises. It uses indexed rendering with an element array buffer to efficiently render meshes.

The loader also supports textured models and automatically computes normals when they are missing from the file.

4.4 Tessellation

The terrain is divided into chunks arranged in a 3×3 grid around the player.

Chunks that are far away from the player do not require a high level of detail because they are barely visible. To optimize rendering performance, we use tessellation to dynamically increase terrain resolution only for nearby chunks.

This technique allows us to send a single quad to the GPU while adapting the terrain resolution according to the player's distance.

4.5 Procedural Tree

The tree defended by the player is procedurally generated using an L-system inspired by the technique described in the book.

First, we implemented the L-system grammar. The generated string is then parsed into segments representing the branches of the tree.

Instead of generating cylinders on the CPU, the segments are sent directly to a geometry shader. The shader emits vertices around the segment extremities to generate cylindrical branches using triangle strips.

To ensure smooth branch connections, the orientation of the parent branch is taken into account. The radius of each branch progressively decreases with the distance from the root.

When the radius becomes sufficiently small, the branch is considered a leaf. The geometry shader generates flattened ellipses instead of perfect cylinders, making the leaves appear larger and more natural.

Finally, the fragment shader applies different textures depending on the branch radius.